

Remove large .pack file created by Git

Asked 14 years, 3 months ago Modified 1 year, 4 months ago Viewed 258k times



I checked a load of files into a branch and merged. And then I had to remove them and now I'm left with a large *.pack* file that I don't know how to get rid of.

213



I deleted all the files using `git rm -rf xxxxxx`, and I also ran the `--cached` option as well.

How can I remove a large *.pack* file that is currently in the following directory?



`.git/objects/pack/pack-xxxxxxxxxxxxxxxxxx.pack`



Do I just need to remove the branch that I still have, but I am no longer using? Or is there something else I need to run?

I'm not sure how much difference it makes but it shows a padlock against the file.

Here are some excerpts from my *bash_history* file that should give an idea how I managed to get into this state (assume at this point I'm working on a Git branch called 'my-branch', and I've got a folder containing more folders/files):

```
git add .
git commit -m "Adding my branch changes to master"
git checkout master
git merge my-branch
git rm -rf unwanted_folder/
rm -rf unwanted_folder/      (I am not sure why I ran this as well, but I did)
```

I thought I also ran the following, but it doesn't appear in the *bash_history* with the others:

```
git rm -rf --cached unwanted_folder/
```

I also thought I ran some Git commands (like `git gc`) to try to tidy up the pack file, but they don't appear in the *.bash_history* file either.

git

branching-and-merging

pack

Share Improve this question Follow

edited May 18, 2025 at 22:02



Peter Mortensen

30.9k 22 112 135

asked Jun 15, 2012 at 12:02



user1116573

3,097 4 21 27

Can you clarify how you removed them? If they are still in the commit history, then they's still be in your pack files. – [loganfsmyth](#) Jun 15, 2012 at 18:36

Hi @loganfsmyth, I've added the bash history scripts that will hopefully help. – [user1116573](#) Jun 16, 2012 at 9:17

Related: [Make Git consume less disk space](#) and [Reduce Git repository size](#) – [Peter Mortensen](#) Oct 24, 2023 at 5:19

7 Answers

Sorted by: Highest score (default) ▾



282

The issue is that, even though you removed the files, they are still present in previous revisions. That's the whole point of Git, is that even if you delete something, you can still get it back by accessing the history.



What you are looking to do is called rewriting history, and it involved the `git filter-branch` command.



GitHub has a good explanation of the issue on their site: [Removing sensitive data from a repository](#)



To answer your question more directly, what you basically need to run is this command with `unwanted_filename_or_folder` replaced accordingly:

```
git filter-branch --index-filter 'git rm -r --cached --ignore-unmatch
unwanted_filename_or_folder' --prune-empty
```

This will remove all references to the files from the active history of the repository.

Next step, to perform a garbage collection cycle to force all references to the file to be expired and purged from the packfile. Nothing needs to be replaced in these commands.

```
git for-each-ref --format='delete %(refname)' refs/original | git update-ref --
stdin
# Or, for older Git versions (e.g. 1.8.3.1) which don't support --stdin
# git update-ref $(git for-each-ref --format='delete %(refname)' refs/original)
git reflog expire --expire=now --all
git gc --aggressive --prune=now
```

Share Improve this answer Follow

edited May 18, 2025 at 22:08



[Peter Mortensen](#)

30.9k 22 112 135

answered Jun 30, 2012 at 21:45



[loganfsmyth](#)

163k 31 350 260

10 Comments ^

Add a comment



[user1116573](#) Over a year ago

I've marked it as accepted if that makes it easier for anyone coming to this question in future, although I actually solved my problem at the time by creating a fresh git repo

▲ 7

Reply



[Damn Vegetables](#) Over a year ago

Why is it this complicated simply to undo one faulty commit? Why can't there be a simple method like `git undo last commit` that deletes all the files added by that commit from the repository file storage? All those commands just to remove the files from the hidden storage...

▲ 2

Reply



[loganfsmyth](#) Over a year ago

@DamnVegetables No argument there, you can definitely make the case for an easy "100% delete the most recent commit" option somewhere.

▲ 0

Reply



[Angel Todorov](#) Over a year ago

Hey guys, I am in the same case, however, when I completed all steps and then call `git push origin --force --all`, I have got *error: denying non-fast-forward refs/heads/master (you should pull first)*, but if I pull, then all my changes will vanish. So what shall I do?

▲ 1

Reply



[sneaky](#) Over a year ago

If you want to identify which files are the biggest files, use `git verify-pack -v .git/objects/pack/pack-{id of your packfile}.idx | sort -k 3 -n | tail -n 20` and `git rev-list --objects --all | grep {id of file}`. Source: [Support Acquia](#)

▲ 1

Reply



Scenario A: If your large files were only added to a branch, you don't need to run `git filter-branch`. You just need to delete the branch and run garbage collection:

27



```
git branch -D mybranch
git reflog expire --expire-unreachable=all --all
git gc --prune=all
```



Scenario B: However, it looks like based on your bash history, that you did merge the changes into master. If you haven't shared the changes with anyone (no `git push` yet). The easiest thing would be to reset master back to before the merge with the branch that had the big files. This will eliminate all commits from your branch and all commits made to master after the merge. So you might lose changes -- in addition to the big files -- that you may have actually wanted:

```
git checkout master
git log # Find the commit hash just before the merge
```

```
git reset --hard <commit hash>
```

Then run the steps from the scenario A.

Scenario C: If there were other changes from the branch **or** changes on master after the merge that you want to keep, it would be best to rebase master and selectively include commits that you want:

```
git checkout master
git log # Find the commit hash just before the merge
git rebase -i <commit hash>
```

In your editor, remove lines that correspond to the commits that added the large files, but leave everything else as is. Save and quit. Your master branch should only contain what you want, and no large files. Note that `git rebase` without `-p` will eliminate merge commits, so you'll be left with a linear history for master after `<commit hash>`. This is probably okay for you, but if not, you could try with `-p`, but `git help rebase` says combining `-p` with the `-i` option explicitly is generally not a good idea unless you know what you are doing.

Then run the commands from scenario A.

Share Improve this answer Follow

edited Oct 26, 2016 at 16:10

answered Mar 25, 2015 at 14:26



onlynone

8,637 3 38 62

4 Comments ^

Add a comment

user4400585 Over a year ago

There's a variant of Scenario A [here](#) with, however, an extra unexpected issue.

0 Reply



xrisz Over a year ago

Scenario A solved mine problem, to delete a big amount of temporary pack file. The repository was managed by a build server and it causes unwanted file creation inside the `.git/objects/pack` folder. I could free up valuable GBs from my disk.

0 Reply



Ryan w Over a year ago

For option A, what if there are changes on the branch I do wish to keep?

0 Reply



onlynone Over a year ago

@Ryanw That would be basically Scenario C. If the changes are still just in your branch and not `master`, just checkout (or keep checked out) your branch and do the `git log` to find the commit hash prior to

add the large file followed by `git rm --cached --ignore-unmatch PATH-TO-YOUR-FILE-WITH-SENSITIVE-DATA`. In the web editor remove the file

▲ 0 Reply ...



20



Run the following command, replacing `PATH-TO-YOUR-FILE-WITH-SENSITIVE-DATA` with the path to the file you want to remove, not just its filename. These arguments will:

1. Force Git to process, but not check out, the entire history of every branch and tag
2. Remove the specified file, as well as any empty commits generated as a result
3. Overwrite your existing tags

```
git filter-branch --force --index-filter "git rm --cached --ignore-unmatch PATH-TO-YOUR-FILE-WITH-SENSITIVE-DATA" --prune-empty --tag-name-filter cat -- --all
```

This will forcefully remove all references to the files from the active history of the repo.

Next step, to perform a GC cycle to force all references to the file to be expired and purged from the pack file. Nothing needs to be replaced in these commands.

```
git update-ref -d refs/original/refs/remotes/origin/master
git for-each-ref --format='delete %(refname)' refs/original | git update-ref --stdin
git reflog expire --expire=now --all
git gc --aggressive --prune=now
```

Share Improve this answer Follow

edited Dec 4, 2023 at 19:33



Peter Mortensen

30.9k 22 112 135

answered Jul 11, 2019 at 17:09



Benjamin Wasula

311 2 5

3 Comments ^

Add a comment



Sambit Swain Over a year ago

I followed the above steps, and pushed as `"git push origin --force --all"` and still my remote branches (master, develop and feature/ASD-1010) didn't clean up. When i fresh cloned from remote repo, it .pack files were still present. How can I reflect this clean up to all remote git branches??

▲ 3 Reply ...



PostCodeism Over a year ago

Same @SambitSwain. This didn't actually change the size of my .pack file. Is there a command above that's missing? I ran `git filter-branch --force --index-filter "git rm --cached --ignore-unmatch .git/objects/pack/pack-cb775202a77613add6cdac4f248d12e026d232f7.pack" --prune-empty --tag-name-filter cat -- --all`

▲ 1 Reply ...

zahir Over a year ago

Why delete `refs/remotes/origin/master` and retain/update others? Does this assume that we performed `filter-branch` on `master`? Also, in my case, I needed to `git push --force --all --prune`. So, be **careful** if your server has extra branches that you don't have locally.

▲ 0

Reply



15

As loganfsmyth already stated in [his answer](#), you need to purge Git history because the files continue to exist there even after deleting them from the repository. Official GitHub documentation [recommend BFG](#) which I find easier to use than `filter-branch`:



Deleting files from history



[Download BFG](#) from their website. Make sure you have the Java runtime installed, then create a mirror clone and purge history. Make sure to replace `YOUR_FILE_NAME` with the name of the file you'd like to delete:

```
git clone --mirror git://example.com/some-big-repo.git
java -jar bfg.jar --delete-files YOUR_FILE_NAME some-big-repo.git
cd some-big-repo.git
git reflog expire --expire=now --all && git gc --prune=now --aggressive
git push
```

Delete a folder

The same as above, but use `--delete-folders`

```
java -jar bfg.jar --delete-folders YOUR_FOLDER_NAME some-big-repo.git
```

Other options

BFG also allows for even fancier options (see [documentation](#)) like these:

Remove all files bigger than 100M from history:

```
java -jar bfg.jar --strip-blobs-bigger-than 100M some-big-repo.git
```

Important!

When running BFG, be careful that both `YOUR_FILE_NAME` and `YOUR_FOLDER_NAME` are indeed just file/folder names. **They're not paths**, so something like `foo/bar.jpg` will not work! Instead all

files/folders with the specified name will be removed from repository history, no matter which path or branch they existed.

Share Improve this answer Follow

edited Dec 4, 2023 at 19:32

answered Apr 4, 2018 at 8:18



Peter Mortensen

30.9k 22 112 135



Timo Ernst

16.1k 25 120 177

2 Comments ^

Add a comment



Angel Todorov Over a year ago

I wonder if I want to apply this `bfg` tool to a local git repo, how the command should look like?

▲ 0 Reply ...



Radzor Over a year ago

Can't push after that: `git push --force` failed to push some refs –

▲ 0 Reply ...

One option:

run `git gc` manually to condense a number of pack files into one or a few pack files. This operation is persistent (i.e., the large pack file will retain its compression behavior), so it may be beneficial to compress a repository periodically with `git gc --aggressive`.

Another option is to save the code and `.git` somewhere and then delete the `.git` and start again using this existing code, creating a new Git repository (`git init`).

Share Improve this answer Follow

edited Dec 4, 2023 at 19:25

answered Jun 15, 2012 at 12:36



Peter Mortensen

30.9k 22 112 135



Michael Durrant

97.4k 102 356 541

3 Comments ^

Add a comment



user1116573 Over a year ago

Hi Michael, I tried running `git gc` and got down to just a couple of pack files but the large one is still one of them and I'd just like to get rid of it so that I can backup the folder externally easier (zip before was 1-2Mb, now 55Mb). Unless someone can suggest anything else I think I may have to create a fresh git. I assume this means I'll lose access to the branches that I currently have etc...?

▲ 1 Reply ...



user1116573 Over a year ago

I gave up trying and just deleted the `.git` folder and created a new git repository as you said. I'll consider it a lesson learnt. Thanks Michael.

▲ 2 Reply ...



jml Over a year ago

This doesn't make much sense. Why can't you just tell git to consolidate the current repository and remove the pack files in the process?

▲ 5 Reply ...



9



In case the previous answers didn't solve the query then I found another way.

Simply remove the specific large file from .pack. I had this issue where I checked in a large 2 GB file accidentally. I followed the steps explained in this link: [HowTo: Completely remove a file from Git history](#)



Share Improve this answer Follow



edited May 18, 2025 at 22:10



Peter Mortensen

30.9k 22 112 135

answered Jan 8, 2018 at 18:34



Rishabh Kumar

557 5 15

2 Comments ^

Add a comment



sam Over a year ago

After doing this method will it completely remove the entire history of the project, or will it just remove the specified file.

▲ 2 Reply ...



Kurt Over a year ago

This has additional useful info as it shows how to search for the large file(s) - so if you are not aware which huge file(s) you committed to your repo, check this post

▲ 0 Reply ...



2



This is using [BFG as recommended by GitHub](#), the same as [Timo's answer](#), but with a slight variation since I spent some time looking at the CLI options.

Let's say I pushed images with over 60 MB a long time ago, and I can't really undo the commit. I would simply run the following



```
java -jar /jarfiles/bfg-1.14.0.jar --delete-files '*. {png,jpg,JPG,PNG}'
```



I would then get a suggestion that I should run the following command, which I will do

```
git reflog expire --expire=now --all && git gc --prune=now --aggressive
```

Finally, synchronise the changes to the remote with

```
git push --force
```

You can verify that the pack file size went down with

```
du -sh ./
```

Share Improve this answer Follow

edited Dec 4, 2023 at 19:36



Peter Mortensen

30.9k 22 112 135

answered Nov 22, 2022 at 21:22



usersina

1,985 1 27 43

Comments

Add a comment

Start asking to get answers

Find the answer to your question by asking.

Ask question

Explore related questions

git

branching-and-merging

pack

See similar questions with these tags.